PropertyViewer コンポーネント UI仕様書

1. 概要

1.1 コンポーネントの目的

PropertyViewerは、PetriNetノード（Material/Condition/Result/Transition）のプロパティを構造化して表示するコンポーネントです。基本情報、テンプレートプロパティ、データプロパティをタブで切り替えて表示します。

1.2 使用場面

- PetriNetViewerでPlaceノードをクリックした際のプロパティ表示
- ModalまたはDrawer内でのプロパティ詳細表示
- ノード情報の閲覧

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

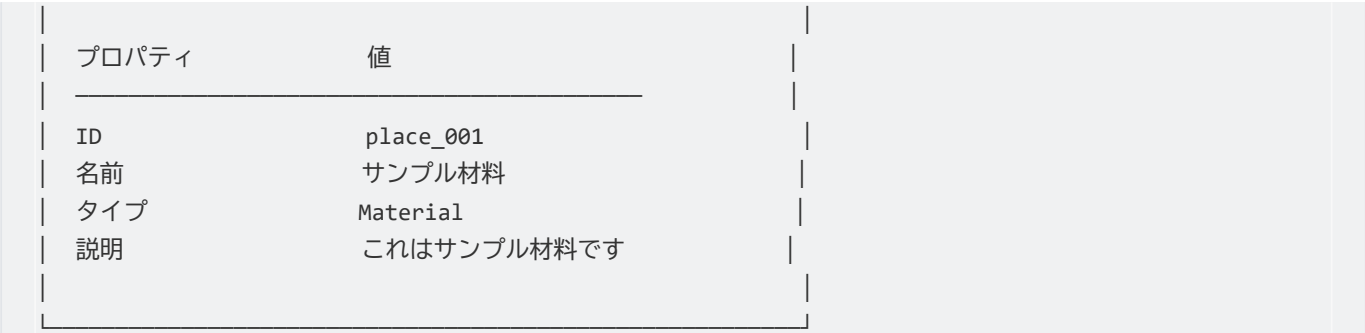
---

2. 画面構成

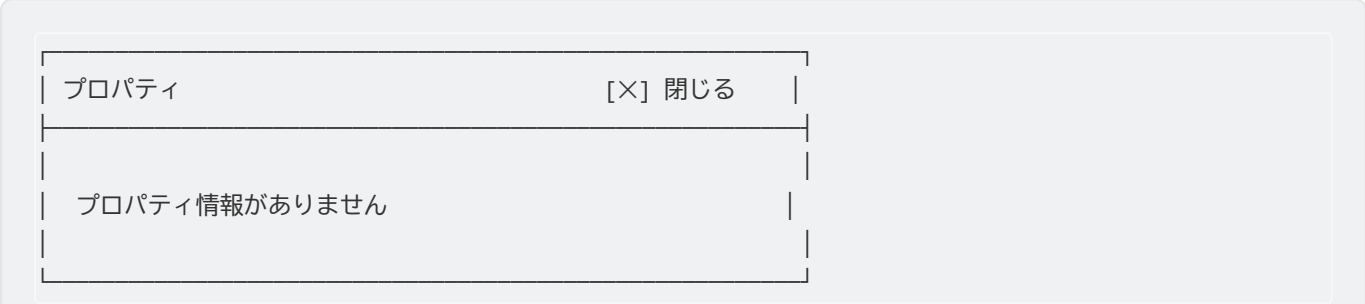
2.1 レイアウト構造

2.1.1 通常表示時

```
┌──────────────────────────────────────────────────────┐
│ [ノード名] プロパティ  [Material]        [×] 閉じる    │ ← ヘッダー（色付き）
├──────────────────────────────────────────────────────┤
│ [基本情報] [テンプレート] [データ]  ← タブ             │
├──────────────────────────────────────────────────────┤
```



2.1.2 プロパティなし



2.2 主要UI要素

2.2.1 ヘッダー

- **背景色**: ノードタイプによって変化
  - **Material**: #F8C471（黄色）
  - **Condition**: #85C1E9（シアン）
  - **Result**: #D7BDE2（マゼンタ）
  - **Transition**: #CCCCCC（グレー）
- **タイトル**: `[ノード名] プロパティ`（Title level 5、白文字）
- **タグ**: ノードタイプ（白背景、ヘッダー色の文字）
- **閉じるボタン**: CloseOutlined アイコン、白文字

2.2.2 タブ

- **基本情報**: ID、名前、タイプ、説明など
- **テンプレート**: テンプレートプロパティ（空の場合は「表示するプロパティがありません」）

- **データ**: データプロパティ（空の場合は「表示するプロパティがありません」）

2.2.3 プロパティテーブル

- **カラム**:
  - プロパティ名（35%幅、太字）
  - 値（自動幅）
- **ヘッダー**: 非表示
- **サイズ**: small
- **ページネーション**: なし

2.2.4 値の表示形式

- **未設定**: "未設定"（グレー、イタリック）
- **真偽値**:
  - `true`: 緑色タグ「はい」
  - `false`: 赤色タグ「いいえ」
- **オブジェクト**: JSON文字列（code形式）
- **その他**: 文字列として表示

---

3. 機能仕様

3.1 構造化プロパティの処理

3.1.1 プロパティ値の取得

```
const getPropertyValue = (propObj: PropertyObject): unknown => {
  if (propObj.value !== undefined) {
    return propObj.value;
  }
  if (propObj.defaultValue !== undefined) {
```

```
    return propObj.defaultValue;
  }
  return '未設定';
};
```

- **優先順位**: value → defaultValue → '未設定'

3.1.2 テーブルデータへの変換

```
const basicInfo = React.useMemo(() => {
  const result: Record<string, unknown> = {};
  Object.entries(structuredProperties.basic || {}).forEach(([key, propObj]) => {
    result[key] = getPropertyValue(propObj);
  });
  return result;
}, [structuredProperties.basic]);
```

- **構造**: `{ basic: {...}, template: {...}, data: {...} }`
- **変換**: PropertyObject → 値のみのRecord

3.2 タブ切り替え

3.2.1 処理フロー

```
const [activeTab, setActiveTab] = useState('basic');

<Tabs
  activeKey={activeTab}
  onChange={setActiveTab}
  items={tabItems}
/>
```

- **初期表示**: 基本情報タブ
- **切り替え**: ユーザークリックで切り替え

3.3 ヘッダー色の動的変更

3.3.1 処理フロー

```
const headerColor = React.useMemo(() => {
  if (!properties?.type) return '#D7BDE2';

  switch (properties.type) {
    case 'Result':
      return '#D7BDE2'; // マゼンタ
    case 'Condition':
      return '#85C1E9'; // シアン
    case 'Material':
      return '#F8C471'; // 黄色
    case 'Transition':
      return '#CCCCCC'; // グレー
    default:
      return '#D7BDE2';
  }
}, [properties?.type]);
```

- **ノードタイプ**: properties.typeに基づく
- **デフォルト**: マゼンタ（#D7BDE2）

3.4 キー名のフォーマット

3.4.1 処理フロー

```
const formatKey = (key: string): string => {
  const keyMap: Record<string, string> = {
    id: 'ID',
    name: '名前',
    type: 'タイプ',
    description: '説明',
    uuid: 'UUID',
    template_uuid: 'Template UUID',
    // ... その他のマッピング
  };
```

```
  return keyMap[key] || key;
};
```

- **機能**: 英語キーを日本語に変換
- **未定義**: キーをそのまま表示

---

4. データ仕様

4.1 Props定義

```typescript
interface PropertyViewerProps {
  properties: {
    id: string;
    name: string;
    type: 'Material' | 'Condition' | 'Result' | 'Transition';
  };
  structuredProperties: StructuredProperties;
  onClose?: () => void;
}
```

Props説明:

Prop名	型	必須	説明
properties	Object	✅	ノードの基本情報（ID、名前、タイプ）
structuredProperties	StructuredProperties	✅	構造化されたプロパティ（Basic/Template/Data）
onClose	Function	-	閉じるボタンクリック時のハンドラ

4.2 型定義

4.2.1 PropertyObject

```typescript
interface PropertyObject {
  key?: string;
  value?: unknown;
```

```
  type?: string;
  description?: string;
  defaultValue?: string;
}
```

説明:

フィールド	型	説明
key	string	プロパティキー
value	unknown	プロパティ値
type	string	データ型
description	string	プロパティの説明
defaultValue	string	デフォルト値

4.2.2 StructuredProperties

```
interface StructuredProperties {
  basic: Record<string, PropertyObject>;
  template: Record<string, PropertyObject>;
  data: Record<string, PropertyObject>;
}
```

説明:

フィールド	型	説明
basic	Record<string, PropertyObject>	基本情報プロパティ
template	Record<string, PropertyObject>	テンプレートプロパティ
data	Record<string, PropertyObject>	データプロパティ

4.3 State定義

```
const [activeTab, setActiveTab] = useState('basic');
```

State説明:

State名	型	初期値	説明
activeTab	string	'basic'	アクティブなタブキー

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
-	（なし）	-	-	-	API呼び出しなし

備考:

- PropertyViewerはPropsとして渡されたデータを表示するのみ
- データ取得は親コンポーネント（PetriNetViewer）で実施

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
propertiesが空またはundefined	「プロパティ情報がありません」メッセージ表示
structuredPropertiesが空	各タブで「表示するプロパティがありません」メッセージ表示

6.2 エラー処理コード

6.2.1 プロパティなし

```
if (!properties || Object.keys(properties).length === 0) {
  return (
    <Card
      title="プロパティ"
```

```
      extra={onClose && <Button type="text" icon={<CloseOutlined />} onClick={onClose} />}
      style={{ height: '100%' }}
    >
      <Text type="secondary">プロパティ情報がありません</Text>
    </Card>
  );
}
```

6.2.2 構造化プロパティなし

```
{
  key: 'template',
  label: 'テンプレート',
  children: Object.keys(templateProps).length > 0 ? (
    <Table ... />
  ) : (
    <Text type="secondary">表示するプロパティがありません</Text>
  ),
}
```

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/dialogs/PropertyViewer.tsx`

7.2 関連ファイル

- なし

7.3 依存コンポーネント

- なし（Ant Designコンポーネントのみ使用）

7.4 依存ライブラリ

- `antd` : Typography, Tabs, Table, Card, Button, Space, Tag, theme
- `@ant-design/icons` : CloseOutlined

---

8. 使用例

8.1 基本的な使用方法

```
import PropertyViewer from '@/components/viewer/dialogs/PropertyViewer';

const structuredProperties = {
  basic: {
    id: { key: 'id', value: 'place_001', type: 'string' },
    name: { key: 'name', value: 'サンプル材料', type: 'string' },
    type: { key: 'type', value: 'Material', type: 'string' },
    description: { key: 'description', value: 'これはサンプル材料です', type: 'string' }
  },
  template: {
    temperature: { key: 'temperature', value: '25°C', type: 'string' },
    pressure: { key: 'pressure', value: '1atm', type: 'string' }
  },
  data: {
    coordinates: { key: 'coordinates', value: '(100, 200)', type: 'string' }
  }
};

<PropertyViewer
  properties={{
    id: 'place_001',
    name: 'サンプル材料',
    type: 'Material'
  }}
  structuredProperties={structuredProperties}
  onClose={() => console.log('閉じる')}
/>
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline